Java & SQL Interview Snippet Questions

Write-on-Whiteboard Style | 3 Years Experience

Short coding questions asked verbally — answer in 2–5 minutes

Every answer shows the clean way + the smart alternative way

Java Core • Java 8 • Collections • OOP • Threads • SQL

[Easy] answer in <3 min **[Medium]** answer in 3-5 min **[Hard]** think before writing

SECTION 1 — Core Java Snippets

Q1–Q15 | Short write-on-spot questions — interviewers hand you a paper and say 'write this'

Q1. [Easy]

Swap Two Variables Without a Third

■ *"Swap a and b without using a temp variable"*

```
int a = 5, b = 10;

// XOR swap
a = a ^ b;
b = a ^ b;
a = a ^ b;
System.out.println("a=" + a + " b=" + b);
```

```
// Output:
a=10 b=5
```

■ Arithmetic way

```
a = a + b;  // a=15
b = a - b;  // b=5
a = a - b;  // a=10
```

■ XOR swap fails when both variables point to the same memory — mention it.

Q2. [Easy]

Check if a Number is Even or Odd Using Bit

■ *"Check even/odd without using % operator"*

```
public static boolean isEven(int n) {
    return (n & 1) == 0;
}
// isEven(4) -> true   isEven(7) -> false
```

```
// Output:
true
false
```

■ Using % (conventional answer)

```
return n % 2 == 0;
```

■ Bit approach (n & 1) is faster — last bit 0 = even, 1 = odd. Show both.

Q3. [Easy]

Find Max of Three Numbers Without Math.max

■ *"Write a method to return max of three integers"*

```
public static int max3(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}
// max3(3, 9, 5) -> 9
```

```
// Output:
9
```

■ One-liner using ternary

```
return a > b ? (a > c ? a : c) : (b > c ? b : c);
```

■ Interviewers sometimes say 'now do it with Math.max' as a follow-up — know both.

Q4. [Easy]

Print Fibonacci Series up to N Terms

■ *"Print first N numbers of Fibonacci series"*

```
public static void fibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        System.out.print(a + " ");
        int c = a + b;
        a = b;
        b = c;
    }
}
// fibonacci(8) -> 0 1 1 2 3 5 8 13
```

```
// Output:
0 1 1 2 3 5 8 13
```

■ Java 8 Stream — impress the interviewer

```
Stream.iterate(new int[]{0,1}, f -> new int[]{f[1], f[0]+f[1]})
      .limit(8).map(f -> f[0]).forEach(n -> System.out.print(n+" "));
```

■ Always lead with iterative (O(1) space), then offer Stream version — shows range.

Q5. [Easy]

Count Digits in a Number

■ *"Count how many digits are in a given integer"*

```
// Method 1 – loop
public static int countDigits(int n) {
    n = Math.abs(n);
    int count = 0;
    do { n /= 10; count++; } while (n > 0);
    return count;
}
// countDigits(12345) -> 5   countDigits(-999) -> 3
```

```
// Output:
5
3
```

■ One-liner

```
return String.valueOf(Math.abs(n)).length();
```

■ do-while handles n=0 correctly (returns 1). if/while would return 0 — common bug.

Q6. [Easy]

Reverse a Number

■ *"Write code to reverse an integer. E.g. 12345 → 54321"*

```
public static int reverseNumber(int n) {
    int reversed = 0;
    boolean negative = n < 0;
    n = Math.abs(n);
    while (n != 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    return negative ? -reversed : reversed;
}
// reverseNumber(12345)  -> 54321
// reverseNumber(-456)   -> -654
```

```
// Output:
54321
-654
```

■ Using StringBuilder

```
new StringBuilder(String.valueOf(Math.abs(n)))
    .reverse().toString()
```

■ Handle negatives and leading zeros (e.g. 1000 → 1). Interviewers always check edge cases.

Q7. [Easy]

Check if a String is Palindrome

■ *"Is 'racecar' a palindrome? Write the check"*

```
public static boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l++) != s.charAt(r--)) return false;
    }
    return true;
}
// isPalindrome("racecar") -> true
// isPalindrome("hello")   -> false
```

```
// Output:
true
false
```

■ One-liner (Java 8)

```
return s.equals(new StringBuilder(s).reverse().toString());
```

■ Two-pointer is O(1) space and exits early — preferred. Show StringBuilder as quick alternative.

Q8. [Medium]

Find All Prime Numbers up to N (Sieve of Eratosthenes)

■ *"Print all primes up to 30"*

```
public static void sieve(int n) {
    boolean[] isComposite = new boolean[n + 1];
    for (int i = 2; i * i <= n; i++) {
        if (!isComposite[i]) {
            for (int j = i*i; j <= n; j += i)
                isComposite[j] = true;
        }
    }
    for (int i=2; i<=n; i++)
        if (!isComposite[i]) System.out.print(i + " ");
}
// sieve(30) -> 2 3 5 7 11 13 17 19 23 29
```

```
// Output:
2 3 5 7 11 13 17 19 23 29
```

■ Simple isPrime check (for single number)

```
boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i=2; i*i<=n; i++)
        if (n%i==0) return false;
    return true;
}
```

■ Sieve is O(n log log n) — much faster than calling isPrime() n times. State the complexity.

Q9. [Easy]

Factorial Using Recursion vs Iteration

■ *"Write factorial — they may ask both ways in same question"*

```
// Iterative (preferred)
public static long factorial(int n) {
    long res = 1;
    for (int i = 2; i <= n; i++) res *= i;
    return res;
}
```

```
// Output:
120  (factorial of 5)
```

■ Recursive (they often ask this next)

```
public static long factRec(int n) {
    return n <= 1 ? 1 : n * factRec(n - 1);
}
```

■ Java 8 — Stream way

```
LongStream.rangeClosed(1, n).reduce(1L, Math::multiplyExact);
```

■ Always say 'iterative is better — no stack overflow risk for large n'. Shows senior maturity.

Q10. [Medium]

Find Duplicate in Array with O(n) Time and O(1) Space

■ *"Array has numbers 1 to n, one number is duplicated. Find it"*

```
// XOR approach — O(n) time, O(1) space
public static int findDuplicate(int[] arr) {
    int xor = 0;
    for (int i = 1; i <= arr.length - 1; i++) xor ^= i;
    for (int n : arr) xor ^= n;
    return xor;
}
// {1,2,3,4,2} -> 2
```

```
// Output:
2
```

■ Sum formula (simpler to explain)

```
int n = arr.length - 1;
int expected = n * (n+1) / 2;
int actual   = Arrays.stream(arr).sum();
return actual - expected; // duplicate
```

■ Sum formula is easier to explain verbally. XOR approach works even when sum might overflow.

Q11. [Easy]

Remove Duplicates from Sorted Array In-Place

■ *"Given sorted array, remove duplicates without extra space"*

```
public static int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;
    int j = 0;
    for (int i = 1; i < arr.length; i++)
        if (arr[i] != arr[j]) arr[++j] = arr[i];
    return j + 1; // new length
}
// {1,1,2,2,3,4,4,5} -> {1,2,3,4,5}, length=5
```

```
// Output:
{1,2,3,4,5}  length=5
```

■ j is the 'write pointer'. Overwrites duplicates in-place. O(1) extra space — LeetCode #26.

Q12. [Easy]

Two Sum — Return Indices of Pair

■ *"Find indices of two numbers that add up to target"*

```
public static int[] twoSum(int[] nums, int target) {
    Map<Integer,Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (map.containsKey(need))
            return new int[]{map.get(need), i};
        map.put(nums[i], i);
    }
    return new int[]{};
}
// {2,7,11,15}, target=9 -> [0,1]
```

```
// Output:
[0, 1]
```

■ Two-pointer (only for sorted array)

```
int l=0, r=nums.length-1;
while (l < r) {
    int sum = nums[l]+nums[r];
    if (sum==target) return new int[]{l,r};
    else if (sum<target) l++; else r--;
}
```

■ HashMap O(n) works on unsorted. Two-pointer O(n) needs sorted array. State the difference.

Q13. [Easy]

Move All Zeros to End (Maintain Order)

■ *"Move zeros to end without changing order of non-zeros"*

```
public static void moveZeros(int[] a) {
    int j = 0;
    for (int i = 0; i < a.length; i++)
        if (a[i] != 0) a[j++] = a[i];
    while (j < a.length) a[j++] = 0;
}
// {0,1,0,3,12} -> {1,3,12,0,0}
```

```
// Output:
{1,3,12,0,0}
```

■ Single pass, two-pointer. O(n) time O(1) space. State it — interviewers are testing complexity awareness.

Q14. [Medium]

Maximum Subarray Sum (Kadane's Algorithm)

■ *"Find the contiguous subarray with maximum sum"*

```
public static int maxSum(int[] arr) {
    int maxSoFar = arr[0], curr = arr[0];
    for (int i = 1; i < arr.length; i++) {
        curr     = Math.max(arr[i], curr + arr[i]);
        maxSoFar = Math.max(maxSoFar, curr);
    }
    return maxSoFar;
}
// {-2,1,-3,4,-1,2,1,-5,4} -> 6  (subarray: 4,-1,2,1)
```

```
// Output:
6
```

■ Kadane's = O(n) time O(1) space. MUST know. If curr goes negative, reset to current element.

Q15. [Easy]

Check if Two Strings are Anagrams

■ *"Are 'listen' and 'silent' anagrams?"*

```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] freq = new int[26];
    for (int i = 0; i < a.length(); i++) {
        freq[a.charAt(i)-'a']++;
        freq[b.charAt(i)-'a']--;
    }
    for (int f : freq) if (f != 0) return false;
    return true;
}
// isAnagram("listen","silent") -> true
```

```
// Output:
true
```

■ **Sort-based (simpler but O(n log n))**

```
char[] ca = a.toCharArray(); Arrays.sort(ca);
char[] cb = b.toCharArray(); Arrays.sort(cb);
return Arrays.equals(ca, cb);
```

■ int[26] approach is O(n) — faster than sorting. Mention both approaches with their complexity.

SECTION 2 — Java 8 & Collections Snippets

Q16–Q28 | 'Write this using Streams' — very common at 2-4 year level

Q16. [Easy]

Find First Non-Repeating Character in String

■ *"Write code to find the first character that appears only once"*

```
public static char firstUnique(String s) {
    Map<Character,Long> freq = s.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(
                c -> c, LinkedHashMap::new, Collectors.counting()));
    return freq.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst().orElse('\0');
}
// firstUnique("swiss") -> w
```

```
// Output:
w
```

■ Traditional (also acceptable, clearer)

```
Map<Character,Integer> m = new LinkedHashMap<>();
for (char c : s.toCharArray()) m.merge(c, 1, Integer::sum);
for (Map.Entry<Character,Integer> e : m.entrySet())
    if (e.getValue()==1) return e.getKey();
return '\0';
```

■ LinkedHashMap preserves insertion order — critical for 'first' requirement. Don't use HashMap.

Q17. [Easy]

Find Second Highest Number in a List

■ *"Given a list of integers, find the second largest"*

```
List<Integer> nums = List.of(5, 1, 9, 3, 7, 9, 2);

Optional<Integer> second = nums.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

System.out.println(second.orElse(-1)); // 7
```

```
// Output:
7
```

■ Without streams — single pass O(n)

```
int max=Integer.MIN_VALUE, second=Integer.MIN_VALUE;
for (int n : nums) {
    if (n > max) { second=max; max=n; }
    else if (n > second && n != max) second=n;
}
```

■ distinct() before sorted() is key — handles duplicates. Single-pass is O(n), always mention it.

Q18. [Easy]

Count Word Frequency in a Sentence

■ *"Count how many times each word appears"*

```
String sentence = "to be or not to be that is the question to";

Map<String,Long> freq = Arrays.stream(sentence.split(" "))
    .collect(Collectors.groupingBy(
             s -> s, Collectors.counting()));

// Sort by frequency descending
freq.entrySet().stream()
    .sorted(Map.Entry.<String,Long>comparingByValue().reversed())
    .forEach(e -> System.out.println(e.getKey()+" : "+e.getValue()));
```

```
// Output:
to : 3
be : 2
or : 1  ...
```

■ groupingBy + counting is the go-to. Follow-up: 'now sort by frequency' — show comparingByValue.

Q19. [Easy]

Filter Employees with Salary > 50000

■ *"Using streams, get names of employees earning more than 50k"*

```
List<String> highEarners = employees.stream()
    .filter(e -> e.getSalary() > 50_000)
    .map(Employee::getName)
    .sorted()
    .collect(Collectors.toList());
```

```
// Output:
[Alice, Bob, Charlie]
```

■ **Get average salary of filtered group**

```
OptionalDouble avg = employees.stream()
    .filter(e -> e.getSalary() > 50_000)
    .mapToInt(Employee::getSalary)
    .average();
```

■ Chain filter → map → sorted → collect cleanly. Interviewers check method reference (::) usage.

Q20. [Easy]

Find Duplicate Elements in a List

■ *"Given a list, find which elements appear more than once"*

```
List<Integer> list = List.of(1,2,3,2,4,3,5,1);

Set<Integer> seen = new HashSet<>();
List<Integer> duplicates = list.stream()
    .filter(n -> !seen.add(n))
    .distinct()
    .collect(Collectors.toList());
// [2, 3, 1]
```

```
// Output:
[2, 3, 1]
```

■ **groupingBy approach**

```
 list.stream()
     .collect(Collectors.groupingBy(n->n, Collectors.counting()))
     .entrySet().stream()
     .filter(e -> e.getValue() > 1)
     .map(Map.Entry::getKey)
     .collect(Collectors.toList());
```

■ !seen.add(n) trick — Set.add() returns false when element already exists. Elegant one-liner.

Q21. [Easy]

Group Employees by Department

■ *"Group a list of employees by their department"*

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Count per department
Map<String, Long> count = employees.stream()
    .collect(Collectors.groupingBy(
             Employee::getDepartment, Collectors.counting()));

// Highest salary per department
Map<String, Optional<Employee>> topEarner = employees.stream()
    .collect(Collectors.groupingBy(
             Employee::getDepartment,
             Collectors.maxBy(Comparator.comparingInt(
                             Employee::getSalary))));
```

■ Know groupingBy with 3 downstream collectors: counting(), averaging(), maxBy(). All 3 are asked.

Q22. [Easy]

Partition List into Even and Odd

■ *"Split a list of numbers into even and odd groups"*

```
List<Integer> nums = List.of(1,2,3,4,5,6,7,8,9,10);

Map<Boolean,List<Integer>> partitioned =
    nums.stream()
        .collect(Collectors.partitioningBy(n -> n%2 == 0));

List<Integer> evens = partitioned.get(true);  // [2,4,6,8,10]
List<Integer> odds  = partitioned.get(false); // [1,3,5,7,9]
```

```
// Output:
evens=[2,4,6,8,10]
odds=[1,3,5,7,9]
```

■ partitioningBy always gives exactly 2 keys (true/false). Simpler than groupingBy for binary split.

Q23. [Easy]

FlatMap — Flatten Nested List

■ *"Flatten List> into a single List"*

```
List<List<Integer>> nested = List.of(
    List.of(1,2,3), List.of(4,5), List.of(6,7,8));

List<Integer> flat = nested.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8]

// Sum of all nested elements
int sum = nested.stream()
    .flatMapToInt(l -> l.stream().mapToInt(Integer::intValue))
    .sum(); // 36
```

```
// Output:
[1,2,3,4,5,6,7,8]
sum=36
```

■ flatMap is the #1 asked Stream method after filter/map. Show flatMapToInt for primitives.

Q24. [Easy]

Convert List to Map Using Streams

■ *"Convert List to Map (id -> name)"*

```
Map<Integer,String> idToName = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Employee::getName));

// Handle duplicate keys — merge function
Map<String,Integer> deptCount = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        e -> 1,
        Integer::sum));  // sum on duplicate key
```

■ Always ask: 'can there be duplicate keys?' — if yes, provide merge function or toMap throws.

Q25. [Easy]

Sort Map by Value

■ *"Sort a HashMap by its values in descending order"*

```
Map<String,Integer> scores = Map.of("Alice",85,"Bob",92,"Carol",78);

scores.entrySet().stream()
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
    .forEach(e ->
        System.out.println(e.getKey()+" -> "+e.getValue()));
```

```
// Output:
Bob -> 92
Alice -> 85
Carol -> 78
```

■ Collect into LinkedHashMap (preserves sorted order)

```
Map<String,Integer> sorted = scores.entrySet().stream()
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (a,b)->a, LinkedHashMap::new));
```

■ LinkedHashMap::new as 4th arg preserves insertion order — always use this when you need a sorted Map.

Q26. [Easy]

Remove Null Values from a List

■ *"Given a list with some nulls, return a clean list"*

```
List<String> names = Arrays.asList(
    "Alice", null, "Bob", null, "Carol");

List<String> clean = names.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
// ["Alice", "Bob", "Carol"]
```

```
// Output:
[Alice, Bob, Carol]
```

■ Also remove empty strings

```
.filter(s -> s != null && !s.isBlank())
```

■ Objects::nonNull as method reference is cleaner than n -> n != null. Know both.

Q27. [Easy]

Find Max and Min in a List

■ *"Find largest and smallest number in a list using streams"*

```
List<Integer> nums = List.of(3,1,7,2,9,4,6);

int max = nums.stream().mapToInt(Integer::intValue).max().getAsInt();
int min = nums.stream().mapToInt(Integer::intValue).min().getAsInt();

// Or using Comparator
Optional<Integer> maxOpt = nums.stream().max(Comparator.naturalOrder());
```

```
// Output:
max=9
min=1
```

■ Using Collections (non-stream)

```
int max = Collections.max(nums);
int min = Collections.min(nums);
```

■ mapToInt avoids autoboxing. IntStream's max()/min() returns OptionalInt — don't forget .getAsInt().

Q28. [Easy]

Check if All/Any/None Match a Condition

■ *"Check: are all numbers positive? Is any number > 100?"*

```
List<Integer> nums = List.of(5, 10, 15, 20, -1, 30);

boolean allPositive = nums.stream().allMatch(n -> n > 0);  // false
boolean anyOverHund = nums.stream().anyMatch(n -> n > 100); // false
boolean noneNegative= nums.stream().noneMatch(n -> n < 0);  // false

// "Are all employees in Engineering dept?
boolean allEng = employees.stream()
    .allMatch(e -> e.getDept().equals("Engineering"));
```

```
// Output:
false
false
false
```

■ allMatch/anyMatch/noneMatch short-circuit — they stop as soon as result is determined. O(n) worst case.

SECTION 3 — OOP & Design Snippets

Q29–Q35 | Design questions — write the class structure on whiteboard

Q29. [Medium]

Implement Immutable Class in Java

■ *"Write an immutable Employee class"*

```
public final class Employee {         // 1. final class
    private final int id;             // 2. final fields
    private final String name;
    private final List<String> skills; // mutable field

    public Employee(int id, String name, List<String> skills) {
        this.id     = id;
        this.name   = name;
        // 3. defensive copy of mutable objects
        this.skills = List.copyOf(skills);
    }
    public int getId()            { return id; }
    public String getName()        { return name; }
    public List<String> getSkills(){ return skills; } // already unmodifiable
    // 4. No setters
}
```

■ 5 rules: final class, private final fields, no setters, defensive copy, initialize in constructor.

Q30. [Medium]

Singleton Pattern — Thread-Safe

■ *"Write a thread-safe Singleton class"*

```
// Best approach – Bill Pugh / Initialization-on-demand
public class DatabaseConnection {
    private DatabaseConnection() {}

    private static class Holder {
        static final DatabaseConnection INSTANCE =
            new DatabaseConnection();
    }
    public static DatabaseConnection getInstance() {
        return Holder.INSTANCE;
    }
}
```

■ **Double-checked locking (also acceptable)**

```
private static volatile DatabaseConnection instance;
public static DatabaseConnection getInstance() {
    if (instance==null) {
        synchronized(DatabaseConnection.class) {
            if (instance==null) instance = new DatabaseConnection();
        }
    }
    return instance;
}
```

■ Bill Pugh: inner class loaded only on first getInstance() call — lazy + thread-safe with no sync overhead.

Q31. [Medium]

Strategy Pattern — Payment System

■ *"Design a payment system where payment method can be swapped at runtime"*

```
// 1. Strategy interface
interface PaymentStrategy {
    void pay(double amount);
}

// 2. Concrete strategies
class CreditCard implements PaymentStrategy {
    public void pay(double amt) {
        System.out.println("Paid "+amt+" via Credit Card"); }
}
class UPI implements PaymentStrategy {
    public void pay(double amt) {
        System.out.println("Paid "+amt+" via UPI"); }
}

// 3. Context
class Checkout {
    private PaymentStrategy strategy;
    public Checkout(PaymentStrategy s) { this.strategy = s; }
    public void complete(double amt)  { strategy.pay(amt); }
}

// Usage
new Checkout(new UPI()).complete(999.0); // Paid 999.0 via UPI
```

```
// Output:
Paid 999.0 via UPI
```

■ In Spring Boot, inject Map to select by name — the production-grade version.

Q32. [Medium]

Builder Pattern — Avoid Constructor with 10 Params

■ *"Create a Person object using Builder pattern"*

```
public class Person {
    private final String name;
    private final int age;
    private final String email;
    private final String phone;

    private Person(Builder b) {
        this.name=b.name; this.age=b.age;
        this.email=b.email; this.phone=b.phone;
    }

    public static class Builder {
        private String name; private int age;
        private String email; private String phone;

        public Builder name(String n)  { this.name=n;  return this; }
        public Builder age(int a)      { this.age=a;   return this; }
        public Builder email(String e) { this.email=e; return this; }
        public Person build()          { return new Person(this); }
    }
}
// Usage
Person p = new Person.Builder().name("Pratiksha").age(25).build();
```

■ In real projects use Lombok @Builder — but always know the manual implementation for interviews.

Q33. [Medium]

Override equals() and hashCode() Correctly

■ *"Write equals and hashCode for an Employee class with id and name"*

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee e)) return false;
    return this.id == e.id &&
           Objects.equals(this.name, e.name);
}

@Override
public int hashCode() {
    return Objects.hash(id, name);
}

// Verify: equal objects must have equal hashCodes
Employee e1 = new Employee(1, "Alice");
Employee e2 = new Employee(1, "Alice");
e1.equals(e2);            // true
e1.hashCode()==e2.hashCode(); // true (contract satisfied)
```

```
// Output:
true
true
```

■ Contract: equals true → hashCode must be same. Break this and HashMap/HashSet will behave incorrectly.

Q34. [Easy]

Demonstrate Polymorphism — Method Overriding

■ *"Show runtime polymorphism with an Animal hierarchy"*

```
abstract class Animal {
    abstract void sound();
    void breathe() { System.out.println("Breathes air"); }
}

class Dog extends Animal {
    void sound() { System.out.println("Woof"); }
}
class Cat extends Animal {
    void sound() { System.out.println("Meow"); }
}

// Runtime polymorphism
List<Animal> animals = List.of(new Dog(), new Cat());
animals.forEach(Animal::sound); // Woof, Meow
```

```
// Output:
Woof
Meow
```

■ Key: reference type is Animal, runtime type is Dog/Cat. JVM resolves method at runtime — that's polymorphism.

Q35. [Easy]

Functional Interface + Lambda — Custom Validator

■ *"Write a custom functional interface and use it with lambda"*

```
@FunctionalInterface
interface Validator<T> {
    boolean validate(T value);
    // only ONE abstract method allowed
}

// Using lambdas
Validator<String> notEmpty   = s -> !s.isEmpty();
Validator<Integer> positive  = n -> n > 0;
Validator<String> emailValid = s -> s.contains("@");

// Chain validators
boolean valid = notEmpty.validate("hello") &&
                emailValid.validate("a@b.com");
System.out.println(valid); // true
```

```
// Output:
true
```

■ @FunctionalInterface annotation is optional but recommended — compiler enforces single abstract method.

SECTION 4 — Multithreading Snippets

Q36–Q42 | Written coding questions asked at 3-year level

Q36. [Easy]

Create Thread — Runnable vs Thread Class

■ *"Show two ways to create a thread in Java"*

```
// Way 1 — Runnable (preferred)
Runnable task = () -> {
    System.out.println("Running: " + Thread.currentThread().getName());
};
new Thread(task, "WorkerThread").start();

// Way 2 — Extend Thread (less preferred)
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// Way 3 — ExecutorService (production way)
ExecutorService pool = Executors.newFixedThreadPool(5);
pool.submit(() -> processTask());
pool.shutdown();
```

```
// Output:
Running: WorkerThread
```

■ Always prefer Runnable over extending Thread — Java allows only single inheritance. Say this.

Q37. [Medium]

Print 1–10 Alternately Using Two Threads

■ *"Thread 1 prints odd, Thread 2 prints even — output: 1 2 3 4 5..."*

```
class OddEven {
    int num = 1; final int MAX = 10;
    final Object lock = new Object();

    void printOdd() throws InterruptedException {
        synchronized(lock) {
            while (num <= MAX) {
                while (num<=MAX && num%2==0) lock.wait();
                if (num<=MAX) System.out.print(num++ + " ");
                lock.notifyAll();
            }
        }
    }
    void printEven() throws InterruptedException {
        synchronized(lock) {
            while (num <= MAX) {
                while (num<=MAX && num%2!=0) lock.wait();
                if (num<=MAX) System.out.print(num++ + " ");
                lock.notifyAll();
            }
        }
    }
}
```

```
// Output:
1 2 3 4 5 6 7 8 9 10
```

■ Most-asked multithreading coding question. Practice until you can write it without thinking.

Q38. [Medium]

Demonstrate Volatile vs Synchronized

■ *"When would you use volatile vs synchronized?"*

```
// volatile — visibility only (flag pattern)
class Worker {
    private volatile boolean running = true;
    void run() {
        while (running) processNextItem();
    }
    void stop() { running = false; } // visible instantly
}

// synchronized — visibility + atomicity
class Counter {
    private int count = 0;
    synchronized void increment() { count++; }
    synchronized int getCount()   { return count; }
}

// AtomicInteger — best for counters (lock-free)
AtomicInteger atomicCount = new AtomicInteger();
atomicCount.incrementAndGet(); // CAS — no lock needed
```

■ volatile for flags/booleans. AtomicInteger for counters. synchronized for complex multi-step ops.

Q39. [Medium]

Producer-Consumer Using BlockingQueue

■ *"Simplest correct producer-consumer without wait/notify"*

```
BlockingQueue<String> queue = new LinkedBlockingQueue<>(5);

// Producer
Thread producer = new Thread(() -> {
    try {
        for (String item : items) { queue.put(item); // blocks if full
            System.out.println("Produced: " + item); }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

// Consumer
Thread consumer = new Thread(() -> {
    try {
        while (true) {
            String item = queue.take(); // blocks if empty
            System.out.println("Consumed: " + item);
        }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
```

■ BlockingQueue IS the right answer — don't manually write wait/notify unless specifically asked.

Q40. [Medium]

CompletableFuture — Run Two Tasks in Parallel

■ *"Fetch user and fetch orders simultaneously, combine results"*

```
CompletableFuture<User>       userFuture  =
    CompletableFuture.supplyAsync(() -> userService.fetch(id));

CompletableFuture<List<Order>> orderFuture =
    CompletableFuture.supplyAsync(() -> orderService.findByUser(id));

// Combine both when done
CompletableFuture<UserProfile> combined =
    userFuture.thenCombine(orderFuture,
        (user, orders) -> new UserProfile(user, orders));

UserProfile profile = combined.get(5, TimeUnit.SECONDS);
```

■ thenCombine runs both futures concurrently — much faster than sequential calls. Key Spring Boot pattern.

Q41. [Medium]

ThreadLocal — Per-Thread Data

■ *"Store user context per thread without passing it everywhere"*

```
public class UserContext {
    private static final ThreadLocal<String> currentUser =
        new ThreadLocal<>();

    public static void set(String user)   { currentUser.set(user); }
    public static String get()             { return currentUser.get(); }
    public static void clear()             { currentUser.remove(); }
}

// In Spring filter
UserContext.set(jwtUtil.extractUser(token));
try { chain.doFilter(req, res); }
finally { UserContext.clear(); } // MUST clear to avoid leak
```

■ ALWAYS clear ThreadLocal in finally block — thread pools reuse threads, stale data causes security bugs.

Q42. [Easy]

Callable vs Runnable — Return a Value from Thread

■ *"Write a thread that computes and returns a value"*

```
// Runnable — no return value
Runnable r = () -> System.out.println("Done");

// Callable — returns a value, can throw checked exception
Callable<Integer> task = () -> {
    Thread.sleep(1000);
    return 42;
};

ExecutorService pool = Executors.newSingleThreadExecutor();
Future<Integer> future = pool.submit(task);
System.out.println(future.get()); // 42 (blocks until done)
pool.shutdown();
```

```
// Output:
42
```

■ Runnable = fire-and-forget. Callable = fire-and-get-result. Future.get() blocks — always use timeout.

SECTION 5 — SQL Snippet Questions

SQL 1–15 | Write the query on paper — most common at 2-4 year Java developer interviews

SQL 1. [Easy]

Find Second Highest Salary

■ *"Write a query to find the second highest salary from Employee table"*

```
-- Method 1: LIMIT + OFFSET (MySQL/PostgreSQL)
SELECT DISTINCT salary
FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

```
// Output:
85000
```

■ **Method 2: Subquery (works everywhere)**

```
SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

■ DISTINCT avoids duplicates skewing the result. Know both — interviewers ask 'another way?'

SQL 2. [Medium]

Find Nth Highest Salary

■ *"Generic query for Nth highest salary"*

```
-- Using DENSE_RANK (best approach)
SELECT salary FROM (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) ranked
WHERE rnk = 3;  -- change 3 for Nth
```

■ **Using LIMIT OFFSET (MySQL/PostgreSQL)**

```
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET N-1;  -- replace N
```

■ DENSE_RANK handles ties correctly. ROW_NUMBER skips ties. Know the difference — often asked.

SQL 3. [Easy]

Find Duplicate Rows in a Table

■ *"Find employees whose name appears more than once"*

```
SELECT name, COUNT(*) AS cnt
FROM employee
GROUP BY name
HAVING COUNT(*) > 1;
```

```
// Output:
Alice | 2
Bob | 3
```

■ HAVING filters AFTER grouping. WHERE filters BEFORE grouping. Classic confusion — state it clearly.

SQL 4. [Easy]

Department-wise Highest Salary

■ *"Get the highest salary in each department"*

```
SELECT department, MAX(salary) AS max_salary
FROM employee
GROUP BY department
ORDER BY max_salary DESC;
```

```
// Output:
Engineering | 120000
HR | 75000
```

■ With employee name (window function)

```
SELECT name, department, salary FROM (
    SELECT *, RANK() OVER
        (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM employee) t
WHERE rnk = 1;
```

■ Simple MAX+GROUP BY gets highest value. Window function gets the employee WITH the highest salary.

SQL 5. [Medium]

Find Employees Who Earn More Than Their Manager

■ *"Self join — employee and manager in same table"*

```
SELECT e.name AS employee, e.salary,
       m.name AS manager,  m.salary AS mgr_salary
FROM   employee e
JOIN   employee m ON e.manager_id = m.id
WHERE  e.salary > m.salary;
```

```
// Output:
Alice | 90000 | Bob | 80000
```

■ Self-join is a very frequently asked SQL pattern. Table aliased twice — e (employee) and m (manager).

SQL 6. [Medium]

DELETE Duplicate Rows, Keep One

■ *"Remove duplicate employee records keeping only one"*

```
-- Keep lowest id for each name (MySQL)
DELETE FROM employee
WHERE id NOT IN (
    SELECT MIN(id) FROM employee GROUP BY name
);
```

■ PostgreSQL — using ctid

```
DELETE FROM employee
WHERE ctid NOT IN (
    SELECT MIN(ctid) FROM employee GROUP BY name);
```

■ Always clarify which duplicate to keep (first/last) — interviewers want you to ask this.

SQL 7. [Easy]

Find All Employees Who Joined in Last 30 Days

■ *"Date filter query"*

```
-- MySQL
SELECT * FROM employee
WHERE join_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);

-- PostgreSQL
SELECT * FROM employee
WHERE join_date >= CURRENT_DATE - INTERVAL '30 days';

-- SQL Server
SELECT * FROM employee
WHERE join_date >= DATEADD(DAY, -30, GETDATE());
```

■ Know date functions for at least MySQL and PostgreSQL — interviewers check if you know the DB-specific syntax.

SQL 8. [Medium]

Count Employees Per Department (Including Empty Depts)

■ *"LEFT JOIN to include departments with zero employees"*

```
SELECT d.name, COUNT(e.id) AS employee_count
FROM   department d
LEFT JOIN employee e ON d.id = e.dept_id
GROUP BY d.name
ORDER BY employee_count DESC;
```

```
// Output:
Engineering | 12
HR | 5
Legal | 0
```

■ LEFT JOIN shows departments with 0 employees. INNER JOIN would hide them. Classic trap question.

SQL 9. [Easy]

Find Customers Who Never Placed an Order

■ *"NOT IN / NOT EXISTS / LEFT JOIN pattern"*

```
-- Method 1: LEFT JOIN (fastest on large tables)
SELECT c.name FROM customer c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

■ Method 2: NOT EXISTS

```
SELECT name FROM customer c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

■ NOT IN fails silently if subquery returns any NULL — NOT EXISTS or LEFT JOIN are safer.

SQL 10. [Medium]

Rank Employees by Salary (RANK vs DENSE_RANK vs ROW_NUMBER)

■ *"Show the difference between 3 ranking functions"*

```
SELECT name, salary,
    ROW_NUMBER()  OVER (ORDER BY salary DESC) AS row_num,
    RANK()        OVER (ORDER BY salary DESC) AS rnk,
    DENSE_RANK()  OVER (ORDER BY salary DESC) AS dense_rnk
FROM employee;

-- Example output with ties at 90000:
-- Alice  | 100000 | 1 | 1 | 1
-- Bob    |  90000 | 2 | 2 | 2
-- Carol  |  90000 | 3 | 2 | 2  <- same RANK and DENSE_RANK
-- Dave   |  80000 | 4 | 4 | 3  <- RANK skips 3, DENSE_RANK doesn't
```

■ RANK skips numbers after ties. DENSE_RANK doesn't skip. ROW_NUMBER is always unique. Know all 3.

SQL 11. [Medium]

Running Total / Cumulative Sum

■ *"Calculate running total of salary ordered by id"*

```
SELECT id, name, salary,
    SUM(salary) OVER (ORDER BY id) AS running_total
FROM employee;

-- Running total per department
SELECT id, name, department, salary,
    SUM(salary) OVER (
        PARTITION BY department ORDER BY id) AS dept_running_total
FROM employee;
```

■ OVER(ORDER BY) creates a window frame. PARTITION BY resets the running total per group.

SQL 12. [Easy]

Find Records in Table A but Not in Table B (EXCEPT)

■ *"Find product ids that exist in Products but not in Orders"*

```
-- EXCEPT (clean and readable)
SELECT id FROM products
EXCEPT
SELECT product_id FROM orders;
```

■ LEFT JOIN approach

```
SELECT p.id FROM products p
LEFT JOIN orders o ON p.id = o.product_id
WHERE o.product_id IS NULL;
```

■ EXCEPT removes duplicates automatically. EXCEPT ALL keeps them. Know both.

SQL 13. [Medium]

Get Latest Record Per Group

■ *"Get each employee's most recent order"*

```
-- Window function (cleanest)
SELECT customer_id, order_id, order_date, amount
FROM (
    SELECT *,
        ROW_NUMBER() OVER
            (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
    FROM orders
) t
WHERE rn = 1;
```

■ ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...) is the standard pattern for 'latest per group'.

SQL 14. [Easy]

Update Salary by 10% for Specific Department

■ *"Give all Engineering employees a 10% raise"*

```
UPDATE employee
SET    salary = salary * 1.10
WHERE  department = 'Engineering';

-- Verify before updating (SELECT first)
SELECT name, salary, salary * 1.10 AS new_salary
FROM   employee
WHERE  department = 'Engineering';
```

■ Always show SELECT before UPDATE in interviews — shows production discipline. Interviewers respect this.

SQL 15. [Easy]

Find Employees with Same Salary

■ *"Find all pairs of employees earning the same amount"*

```
SELECT a.name, b.name, a.salary
FROM   employee a
JOIN   employee b ON a.salary = b.salary
                  AND a.id < b.id  -- avoid (A,B) and (B,A) duplicates
ORDER BY a.salary DESC;
```

```
// Output:
Alice | Bob | 90000
```

■ a.id < b.id prevents duplicate pairs and self-join. This detail shows SQL maturity.
